Working MCP Server Packages for Claude Desktop on Windows

Five MCP servers for Prisma, WebSocket, Redis, Docker, and Python REPL are available—but only three have official packages. The Prisma server is built into the Prisma CLI, Redis has an official Python package (the npm version is deprecated), Docker and Python REPL use community Python packages via `uvx`, while WebSocket requires a bridging solution since native WebSocket transport isn't yet supported in MCP.

Prisma: Built into the CLI itself

The Prisma MCP server ships directly with the **Prisma CLI** starting from v6.6.0. `Prisma` No separate package installation is needed—just run `npx -y prisma mcp`.

Package: `prisma` (npm)
Type: Node.js
Status: ✅ Officially maintained by Prisma team

Claude Desktop Config (`%APPDATA%\Claude\claude_desktop_config.json`):

```json
{
  "mcpServers": {
    "Prisma": {
      "command": "npx",
      "args": ["-y", "prisma", "mcp"]
    }
  }
}
```

Environment variables: None required. Your project's `DATABASE_URL` in `.env` is used automatically when running migrations and schema commands.

Tools provided: `migrate-status`, `migrate-dev`, `migrate-reset`, `GitHub` `Prisma-Studio`, database creation, and cloud management via optional remote server at `https://mcp.prisma.io/mcp`. `prisma`

WebSocket: No official package exists yet

Important: There is no `@modelcontextprotocol/server-websocket` package. Native WebSocket transport for MCP (SEP-1288) is still under review. The official SDK supports only **stdio** and **Streamable HTTP/SSE** transports. `npm`

Best solution: Use `mcp2websocket` to bridge stdio to WebSocket servers. npm

Package: `mcp2websocket` (npm)
Type: Node.js bridge
Status: ✅ Community maintained, actively updated

Claude Desktop Config:

```
json

{
  "mcpServers": {
    "websocket": {
      "command": "npx",
      "args": ["-y", "mcp2websocket", "ws://localhost:8765/mcp"]
    }
  }
}
```

Environment variables (optional):

- `AUTH_TOKEN` — Authentication token for protected WebSocket servers npm
- `DEBUG` — Set to `"true"` for verbose logging npm

Alternative: `supergateway` converts any stdio MCP server into a WebSocket or SSE service, useful if you need to expose a local server remotely.

Redis: Use the official Python package

The npm package `@modelcontextprotocol/server-redis` is **officially deprecated** as of early 2025. npm Use the official Python-based server maintained by Redis Labs.

Package: `redis-mcp-server` (PyPI)
Type: Python (run via `uvx`)
Status: ✅ Officially maintained by Redis team, v0.3.6 released November 2025

Claude Desktop Config:

```
json
```

```
{
  "mcpServers": {
    "redis": {
      "command": "uvx",
      "args": [
        "--from", "redis-mcp-server@latest",
        "redis-mcp-server",
        "--url", "redis://localhost:6379/0"
      ]
    }
  }
}
```

Alternative using environment variables:

```
json

{
  "mcpServers": {
    "redis": {
      "command": "uvx",
      "args": ["--from", "redis-mcp-server@latest", "redis-mcp-server"],
      "env": {
        "REDIS_HOST": "localhost",
        "REDIS_PORT": "6379",
        "REDIS_PWD": "yourpassword"
      }
    }
  }
}
```

Required environment variables:

Variable	Default	Description
`REDIS_HOST`	`127.0.0.1`	Redis hostname
`REDIS_PORT`	`6379`	Redis port
`REDIS_PWD`	(empty)	Password if required
`REDIS_SSL`	`False`	Enable TLS

Windows note: Redis doesn't run natively on Windows. Use **WSL**, **Docker Desktop**, **Memurai**, or **Redis Cloud**. npm

---

Docker: Community Python package via uvx

No official Docker MCP server exists, but docker-mcp is the most widely adopted community solution with **430+ GitHub stars**.

Package: docker-mcp (PyPI)
Type: Python (run via uvx)
Status: ✅ Actively maintained, v0.2.0 released December 2024
GitHub: github.com/QuantGeekDev/docker-mcp

Claude Desktop Config:

```json
{
  "mcpServers": {
    "docker-mcp": {
      "command": "uvx",
      "args": ["docker-mcp"]
    }
  }
}
```

Environment variables: None required—the server connects to your local Docker daemon automatically via the standard Docker socket.

Tools provided: create-container, deploy-compose, get-logs, list-containers. PyPI GitHub

Prerequisites: Docker Desktop must be running before launching Claude Desktop.

Python REPL: Feature-rich interpreter package

Package: mcp-python-interpreter (PyPI)
Type: Python (run via uvx)
Status: ✅ Actively maintained, 77+ GitHub stars, MIT license
GitHub: github.com/yzfly/mcp-python-interpreter

Claude Desktop Config:

```json
{
  "mcpServers": {
    "python-interpreter": {
      "command": "uvx",
      "args": [
        "mcp-python-interpreter",
        "--dir", "C:\\Users\\YourName\\python-work",
        "--python-path", "C:\\Python311\\python.exe"
      ],
      "env": {
        "MCP_ALLOW_SYSTEM_ACCESS": "0"
      }
    }
  }
}
```

Required parameters:

Parameter	Required	Description
`--dir`	**Yes**	Working directory for file isolation GitHub
`--python-path`	No	Custom Python executable path

Environment variables:

- `MCP_ALLOW_SYSTEM_ACCESS` — Set to `"0"` to restrict system access (recommended)

Tools provided: `run_python_code`, `run_python_file`, `list_python_environments`, `install_package`, `read_file`, `write_file`, `list_directory`. github

Windows prerequisites and setup

Three of these servers require **uvx** (the Python package runner from Astral). Install it first:

```powershell
powershell -ExecutionPolicy Bypass -Command "iwr -useb https://astral.sh/uv/install.ps1 | iex"
```

Alternatively, install via pip: `pip install uv`

Config file location: `%APPDATA%\Claude\claude_desktop_config.json` GitHub GitHub
(Typically `C:\Users\<YourName>\AppData\Roaming\Claude\claude_desktop_config.json`)

Path formatting: Use double backslashes (`\\`) in JSON config files.

Complete combined configuration

Here's a single config file with all five servers:

```json
{
  "mcpServers": {
    "Prisma": {
      "command": "npx",
      "args": ["-y", "prisma", "mcp"]
    },
    "websocket": {
      "command": "npx",
      "args": ["-y", "mcp2websocket", "ws://localhost:8765/mcp"]
    },
    "redis": {
      "command": "uvx",
      "args": ["--from", "redis-mcp-server@latest", "redis-mcp-server", "--url", "redis://localhost:6379/0"]
    },
    "docker-mcp": {
      "command": "uvx",
      "args": ["docker-mcp"]
    },
    "python-interpreter": {
      "command": "uvx",
      "args": ["mcp-python-interpreter", "--dir", "C:\\Users\\YourName\\python-work"],
      "env": {
        "MCP_ALLOW_SYSTEM_ACCESS": "0"
      }
    }
  }
}
```

Summary table

Service	Package Name	Type	Command	Status
Prisma	`prisma`	npm	`npx -y prisma mcp`	✅ Official
WebSocket	`mcp2websocket`	npm	`npx -y mcp2websocket <url>`	⚠️ Bridge (no native)
Redis	`redis-mcp-server`	PyPI	`uvx --from redis-mcp-server@latest redis-mcp-server`	✅ Official
Docker	`docker-mcp`	PyPI	`uvx docker-mcp`	✅ Community (430★)
Python REPL	`mcp-python-interpreter`	PyPI	`uvx mcp-python-interpreter --dir <path>`	✅ Community (77★)

Key insight: The npm-based `@modelcontextprotocol/server-redis` is deprecated— `npm` always use the Python `redis-mcp-server` package. For WebSocket connections, native MCP support is still pending, so use the `mcp2websocket` bridge as a workaround.